

Homework 02

Justin Adrowski

Problem 1:

A) Assume t0 holds the value 0x0000FFF, t1 holds the value 0xEFF01270, t2 holds the value 0x00000001 and t3 holds the value 0x0000001F. What is the value of t2 after the following instructions:

```
lui t0, 0xFFFFF
slt t1, t1, t0
beq t1, zero, SOMEPLACE
sra t2, t1, t3
j EXIT
SOMEPLACE:
    add t2, t2, t1
EXIT:
```

I1, t0 = 0xfffff000

I2, (two's comp) => fffff000 = -4096, eff01270 = -269027728 => t1 < t0, t1 = 0x00000001

I3, t1 != 0, next line

I4, shift t1 to the right by 31 bits, store in t2 => gets rid of 1 bit, t2 = 0x00000000

I5, exit (no someplace block)

B) Translate C loop into RISC-V with following register maps. All vars = 4byte. How many instructions executed if N = 1, 10, 100, 1000?

int *a	s0
int max	s1
int i	s2
int N	s3

```
int max = a[0];
for (i=1; i<N; i++) {
    max = a[i] > max ? a[i] : max;
}
```

```

addi t2, zero, 0
addi t1, zero, 0
lw s1, 0(s0)
addi s2, x0, 1
for_loop:
    bge s2, s3, end_of_for
    slli t4, s2, 2
    add t1, s0, t4
    lw t2, 0(t1)
    slt t3, s1, t2
    beq t3, zero, max_stay
        add s1, x0, t2
    max_stay:
        addi s2, s2, 1
        j for_loop
end_of_for:

```

N = 1 => 5 Instructions

N = 10 => 77 instructions (86 if max updates every time)

N = 100 => 797 instructions (896 if max updates every time)

N = 1000 => 7997 instructions (8996 if max updates every time)

C) More RISCV Assembly from C code

```
if (score >= 90) {  
    grade = 'A';  
} else if (score >= 80) {  
    grade = 'B';  
} else if (score >= 70) {  
    grade = 'C';  
} else if (score >= 60) {  
    grade = 'D';  
} else {  
    grade = 'F';  
}
```

li t0, 90

bge a0, t0, SET_A

li t0, 80

bge a0, t0, SET_B

li t0, 70

bge a0, t0, SET_C

li t0, 60

bge a0, t0, SET_D

j SET_F

SET_A: li t1, 'A'

j DONE

SET_B: li t1, 'B'

j DONE

SET_C: li t1, 'C'

j DONE

SET_D: li t1, 'D'

j DONE

SET_F: li t1, 'F'

DONE:

Problem 2: RISC-V Assembly Language Design

a) Bitwise Invert in RISC-V

```
Xori t1, t2, -1
```

b) Rotate Left immediate (negative immediate implies right rotate)

```
andi t2, t3, 31 # get shift left amount if sign_ext_imm is negative
```

```
li t4, 32
```

```
sub t4, t4, t2
```

```
sll t0, t1, t2
```

```
srl t1, t1, t4
```

```
or t0, t0, t1
```

This implementation doesn't produce any exceptions that I can think of.

c) Labels in RISC-V

1 – there's no room for a full address, let alone three, in a 32-bit instruction.

2 – only the lw and sw instructions can access memory since they only require one memory access port. Having the capability to perform instructions with labels, accessing up to three memory points in one instruction would require a lot more hardware capabilities that would be expensive.

Problem 3: RISC-V Programming

b) C Code implemetaton of strchr

```
C p3.c
1
2 char *strrchr(char *str, int character) {
3     char target = (char) character;
4     char *match = NULL;
5     char current = *str;
6
7     while (1) {
8         if (current == target) {
9             match = str;
10        }
11        if (current == '\0') {
12            break;
13        }
14        str++;
15        current = *str;
16    }
17
18    return match;
19 }
```

b) RISC-V function implementation

```
ASM p3b.s
1 .text
2 .globl strrchr
3
4 ▾ strrchr:
5     andi    a1, a1, 0xFF      -- store lower 8 of target
6     mv      t0, zero         -- reset match storage
7
8 ▾ scan_loop:
9     lbu     t1, 0(a0)         -- get current character
10    bne     t1, a1, check_end  -- compare to target
11    mv      t0, a0            -- copy match location
12 ▾ check_end:
13    beqz    t1, scan_done      -- check for null
14    addi    a0, a0, 1          -- increment index
15    j       scan_loop         -- jump back to start of loop
16
17 ▾ scan_done:
18    mv      a0, t0            -- store match location
19    ret                      -- return
```

c) RISC-V Test Driver

```
asm p3cs
1  .data
2  str_http: .asciz "howdy there partner"
3  str_howdy: .asciz "howdy"
4
5  msg1: .asciz "\nTest 1: strrchr(\"howdy there partner\", 't')
6  -- included 't' because it appears at the end of the string
7  msg2: .asciz "\nTest 2: strrchr(\"howdy there partner\", 'q')
8  -- included 'q' to see if it correctly returns a null
9  msg3: .asciz "\nTest 3: strrchr(\"howdy\", '\\0')
10 -- included null to see if it correctly points to the end of the string
11 sep_msg: .asciz " -> \""
12 close_nl: .asciz "\\n"
13 not_found_msg: .asciz "NULL (character not found)\n"
14
15  .text
16  .globl main
17  main:
18      li    a7, 4
19      la    a0, msg1
20      ecall
21      la    a0, str_http
22      li    a1, 'o'
23      jal   ra, strrchr
24      jal   ra, print_result
25
26      li    a7, 4
27      la    a0, msg2
28      ecall
29      la    a0, str_http
30      li    a1, 'z'
31      jal   ra, strrchr
32      jal   ra, print_result
33
34      li    a7, 4
35      la    a0, msg3
36      ecall
37      la    a0, str_howdy
38      li    a1, 0
39      jal   ra, strrchr
40      jal   ra, print_result
41
42      li    a7, 10
43      ecall
44
45
46  print_result:
47      beqz  a0, pr_null
48
49      mv    t2, a0
50      li    a7, 34
51      ecall
52
53      li    a7, 4
54      la    a0, sep_msg
55      ecall
56
57      li    a7, 4
58      mv    a0, t2
59      ecall
60
61      li    a7, 4
62      la    a0, close_nl
63      ecall
64      ret
65
66  pr_null:
67      li    a7, 4
68      la    a0, not_found_msg
69      ecall
70      ret
```